

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение высшего образования
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ТОМСКИЙ ПОЛИТЕХНИЧЕСКИЙ
УНИВЕРСИТЕТ»



Инженерная школа информационных технологий и робототехники

Отделение информационных технологий

09.04.01 «Информатика и вычислительная техника»

Отчет по лабораторной работе №6
«Реализация U-Net подобной нейронной сети для сегментации изображений
дорожного движения с помощью трансферного обучения»

МЕТОДЫ РАСПОЗНАВАНИЯ ОБРАЗОВ

Исполнитель:

студент группы

8BM22

Ямкин Н.Н.

Руководитель:

Доцент (ОИТ, ИШИТР)

Друки А.А.

Содержание

1. Цель работы	3
2. Задачи	3
3. Ход работы	4
4. Заключение.....	10

1. Цель работы

Подобным образом реализовать U-Net подобную нейронную сеть для сегментации изображений дорожного движения. Изображения имеют несколько классов объектов: люди, дорога, деревья, дома и т.д. Необходимо окрасить каждый класс объекта в определенный цвет.

Сжимающая часть сети будет состоять из импортированных слоев. Расширяющая часть может состоять из ваших оригинальных слоев. Как реализовать сеть UNet можно посмотреть в 5 лабораторной работе по курсу «Машинное обучение».

Набор данных GTA5

Набор данных GTA5 содержит 401 изображение в формате png для обучающего набора и 201 изображение в формате png для валидационного набора. Маска содержит 401 изображение в формате png из обучающего набора и 201 изображение в формате png из валидационного набора.

2. Задачи

1. Загрузить и обработать датасет;
2. Реализовать U-Net подобную нейронную сеть из импортированных слоев и своих оригинальных слоев;
3. Обучить и протестировать созданную нейронную сеть.

3. Ход работы

Создадим новый блокнот в Google Colab и импортируем необходимые для работы с изображениями библиотеки и загрузим сами данные:

```
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image, ImageDraw
from skimage import io
import os

from google.colab import drive
drive.mount('/content/drive')

def download_data(path):
    data = []
    for path_image in sorted(os.listdir(path=path)):
        image = Image.open(path + path_image) # Открываем изображение
        data.append(np.array(image)) # Загружаем пиксели
    return data

X_train = download_data(r"/content/drive/MyDrive/Методы распознавания
образов/Resized images/Images/Train/")
Y_train = download_data(r"/content/drive/MyDrive/Методы распознавания
образов/Resized images/Masks/Train/")
X_test = download_data(r"/content/drive/MyDrive/Методы распознавания
образов/Resized images/Images/Test/")
Y_test = download_data(r"/content/drive/MyDrive/Методы распознавания
образов/Resized images/Masks/Test/")
```

Выведем пример изображений и соответствующих им масок.

```
plt.figure(figsize=(6,4))
for i in range(8):
    plt.subplot(4,4,i+1)
    plt.xticks([])
    plt.yticks([])
    plt.imshow(X_train[i])

plt.figure(figsize=(6,4))
for i in range(8):
    plt.subplot(4,4,i+1)
    plt.xticks([])
    plt.yticks([])
    plt.imshow(Y_train[i])
```

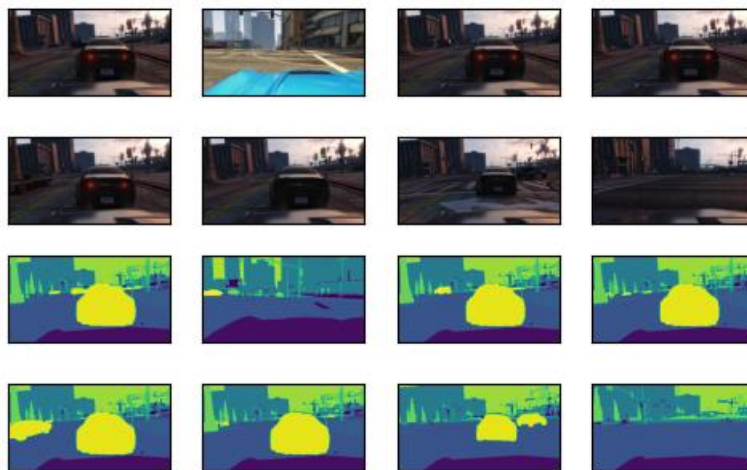


Рисунок 1 – Пример изображений

Обработаем исходный набор данных.

```
X_train_pred = np.array(X_train).reshape([len(X_train), 256, 480, 3])/255
X_test_pred = np.array(X_test).reshape([len(X_test), 256, 480, 3])/255
Y_train_pred = []

Y_train_pred = np.array(Y_train)
Y_test_pred = np.array(Y_test)
```

`X_train_pred` и `X_test_pred` создаются путем преобразования массивов исходных данных `X_train` и `X_test` в массивы NumPy. Затем они переформируются в массивы с размерами `[len(X_train), 256, 480, 3]` и `[len(X_test), 256, 480, 3]`. Это говорит о том, что обрабатываются данные изображения с высотой 256 и шириной 480 с 3 цветовыми каналами (вероятно, RGB). Кроме того, разделим значения на 255, что является стандартным шагом для приведения значений пикселей к диапазону `[0, 1]` и обычно выполняется перед обучением нейронной сети.

`Y_train_pred` и `Y_test_pred` также преобразуются в массивы NumPy из массивов исходных данных `Y_train` и `Y_test`. При этом предполагается, что `Y_train` и `Y_test` представляют собой метки или целевые значения.

Определим архитектуру нейронной сети.

```
vgg_model = VGG16(weights='imagenet', include_top=False, input_shape=(256, 480, 3))

conv1 = vgg_model.get_layer('block1_conv1').output
conv2 = vgg_model.get_layer('block1_conv2') (conv1)
conv3 = vgg_model.get_layer('block1_conv2') (conv2)
batch1 = BatchNormalization() (conv3)
drop1 = Dropout(0.5) (batch1)
block_1 = layers.add([conv1, drop1])
pool1 = vgg_model.get_layer('block1_pool') (block_1)

conv4 = vgg_model.get_layer('block2_conv1') (pool1)
conv5 = vgg_model.get_layer('block2_conv2') (conv4)
conv6 = vgg_model.get_layer('block2_conv2') (conv5)
batch2 = BatchNormalization() (conv6)
drop2 = Dropout(0.5) (batch2)
block_2 = layers.add([conv4, drop2])
pool2 = vgg_model.get_layer('block2_pool') (block_2)

conv7 = vgg_model.get_layer('block3_conv1') (pool2)
conv8 = vgg_model.get_layer('block3_conv2') (conv7)
conv9 = vgg_model.get_layer('block3_conv3') (conv8)
batch3 = BatchNormalization() (conv9)
drop3 = Dropout(0.5) (batch3)
block_3 = layers.add([conv7, drop3])
pool3 = vgg_model.get_layer('block3_pool') (block_3)

conv10 = vgg_model.get_layer('block4_conv1') (pool3)
conv11 = vgg_model.get_layer('block4_conv2') (conv10)
conv12 = vgg_model.get_layer('block4_conv3') (conv11)
batch4 = BatchNormalization() (conv12)
drop4 = Dropout(0.5) (batch4)
block_4 = layers.add([conv10, drop4])
pool4 = vgg_model.get_layer('block4_pool') (block_4)

conv13 = vgg_model.get_layer('block5_conv1') (pool4)
conv14 = vgg_model.get_layer('block5_conv2') (conv13)
conv15 = vgg_model.get_layer('block5_conv3') (conv14)
batch5 = BatchNormalization() (conv15)
drop5 = Dropout(0.5) (batch5)
block_5 = layers.add([conv13, drop5])
pool5 = vgg_model.get_layer('block5_pool') (block_5)

up1 = UpSampling2D(size = (2,2)) (pool5)
mergel = concatenate([conv13, up1], axis = 3)
conv16 = Conv2D(64, 5, activation = 'relu', padding = 'same') (mergel)
conv17 = Conv2D(64, 5, activation = 'relu', padding = 'same') (conv16)
conv18 = Conv2D(64, 5, activation = 'relu', padding = 'same') (conv17)
batch6 = BatchNormalization() (conv18)
```

```

drop6 = Dropout(0.5) (batch6)
block_6 = layers.add([conv16, drop6])

up2 = UpSampling2D(size = (2,2)) (block_6)
merge2 = concatenate([conv10, up2], axis = 3)
conv19 = Conv2D(32, 3, activation = 'relu', padding = 'same') (merge2)
conv20 = Conv2D(32, 3, activation = 'relu', padding = 'same') (conv19)
conv21 = Conv2D(32, 3, activation = 'relu', padding = 'same') (conv20)
batch7 = BatchNormalization() (conv21)
drop7 = Dropout(0.5) (batch7)
block_7 = layers.add([conv19, drop7])

up3 = UpSampling2D(size = (2,2)) (block_7)
merge3 = concatenate([conv7, up3], axis = 3)
conv22 = Conv2D(32, 3, activation = 'relu', padding = 'same') (merge3)
conv23 = Conv2D(32, 3, activation = 'relu', padding = 'same') (conv22)
conv24 = Conv2D(32, 3, activation = 'relu', padding = 'same') (conv23)
batch8 = BatchNormalization() (conv24)
drop8 = Dropout(0.5) (batch8)
block_8 = layers.add([conv22, drop8])

up4 = UpSampling2D(size = (2,2)) (block_8)
merge4 = concatenate([conv4, up4], axis = 3)
conv25 = Conv2D(32, 3, activation = 'relu', padding = 'same') (merge4)
conv26 = Conv2D(32, 3, activation = 'relu', padding = 'same') (conv25)
conv27 = Conv2D(32, 3, activation = 'relu', padding = 'same') (conv26)
batch9 = BatchNormalization() (conv27)
drop9 = Dropout(0.5) (batch9)
block_9 = layers.add([conv25, drop9])

up5 = UpSampling2D(size = (2,2)) (block_9)
merge5 = concatenate([conv1, up5], axis = 3)
conv28 = Conv2D(32, 3, activation = 'relu', padding = 'same') (merge5)
conv29 = Conv2D(32, 3, activation = 'relu', padding = 'same') (conv28)
conv30 = Conv2D(32, 3, activation = 'relu', padding = 'same') (conv29)
batch10 = BatchNormalization() (conv30)
drop10 = Dropout(0.2) (batch10)
block_9 = layers.add([conv28, drop10])

conv31 = Conv2D(35, 3, activation = 'softmax', padding = 'same') (block_9)

model = Model(inputs = vgg_model.input, outputs = conv31)

model.compile(optimizer = 'adam', loss = 'sparse_categorical_crossentropy',
metrics = 'accuracy')

```

Установим все слои модели VGG как необучаемые.

```

for layer in vgg_model.layers:
    layer.trainable = False

```

В глубоком обучении иногда используются предварительно обученные модели для миграционного обучения. При миграционном обучении обычно загружаются веса модели, обученной на крупном наборе данных (например, VGG, ResNet и т.д.), а затем производится тонкая настройка модели под вашу задачу.

Установив для всех слоев значение "необучаемый" (`layer.trainable = False`), вы гарантируете, что веса этих слоев останутся неизменными при последующем обучении и не будут обновляться по мере обучения набора данных. Это связано с желанием сохранить те характеристики, которые эти слои выучили на крупном наборе данных.

Если требуется более тонкая настройка некоторых слоев модели, можно задать их обучаемыми, а остальные оставить неизменными.

Это действие обычно выполняется после загрузки модели, но до начала обучения. Это позволяет избежать изменения весов предварительно обученной модели в процессе обучения.

Обучим и протестируем нейронную сеть.

```
model.fit(X_train_pred, Y_train_pred, batch_size=10, epochs=50)
model.evaluate(X_test_pred, Y_test_pred, batch_size=1)
out = model.predict(np.array(X_test_pred[:100]), batch_size=1)
```

```
Epoch 43/50
41/41 [=====] - 44s 1s/step - loss: 0.3501 - accuracy: 0.8951
Epoch 44/50
41/41 [=====] - 44s 1s/step - loss: 0.3371 - accuracy: 0.8992
Epoch 45/50
41/41 [=====] - 43s 1s/step - loss: 0.3691 - accuracy: 0.8900
Epoch 46/50
41/41 [=====] - 43s 1s/step - loss: 0.3423 - accuracy: 0.8972
Epoch 47/50
41/41 [=====] - 43s 1s/step - loss: 0.3274 - accuracy: 0.9011
Epoch 48/50
41/41 [=====] - 43s 1s/step - loss: 0.3148 - accuracy: 0.9049
Epoch 49/50
41/41 [=====] - 43s 1s/step - loss: 0.3074 - accuracy: 0.9068
Epoch 50/50
41/41 [=====] - 43s 1s/step - loss: 0.3066 - accuracy: 0.9069
<keras.src.callbacks.History at 0x7d9ed7d7a080>
```

Рисунок 2 – Результаты обучения модели

Выведем результат сегментации.

```
I = 1  
plt.imshow(np.array(X_test[I]))  
plt.show()  
plt.imshow(np.array(Y_test[I]))  
plt.show()  
plt.imshow(np.argmax(out[I], axis = -1))  
plt.show()
```

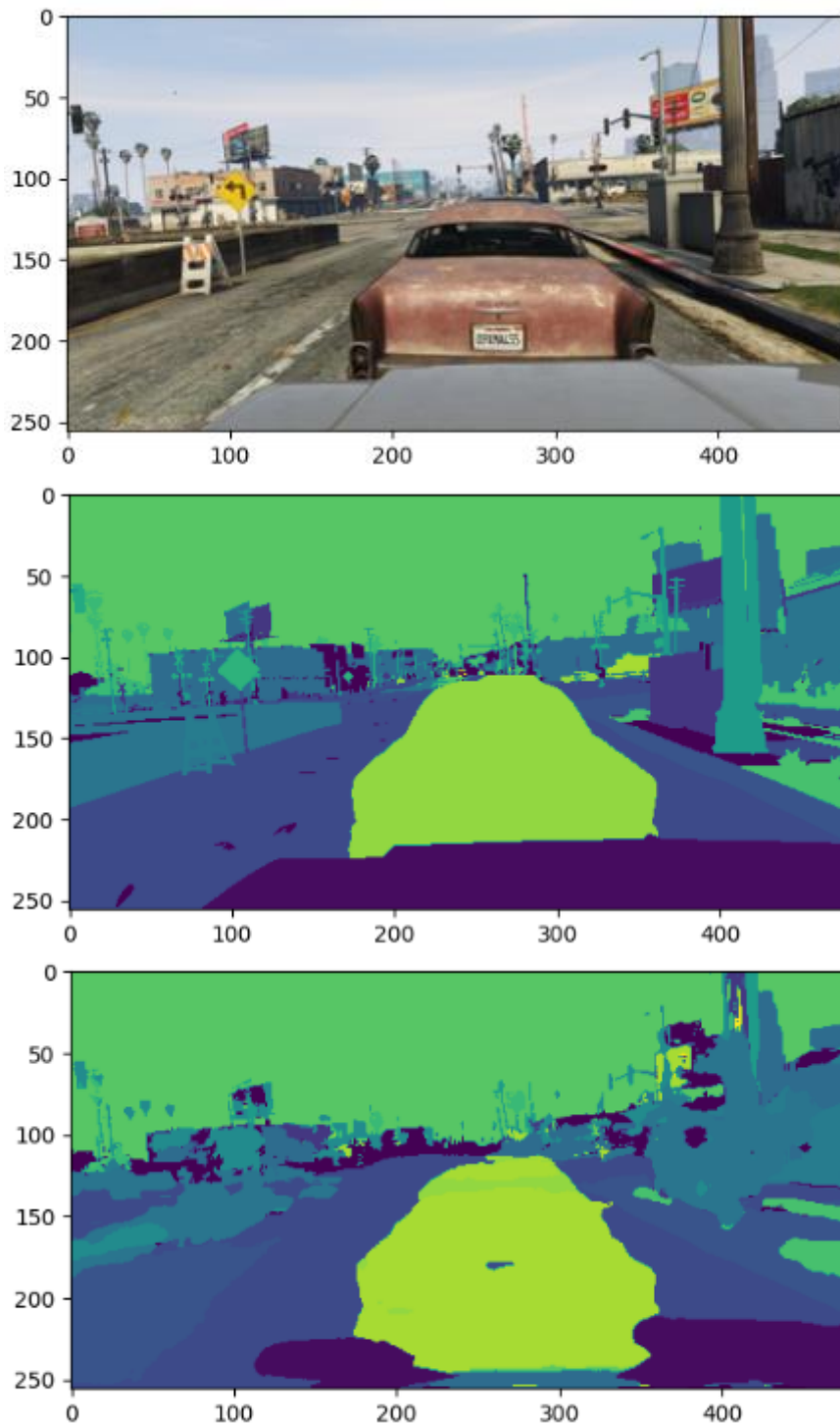


Рисунок 3 – Сегментация изображения

4. Заключение

В результате выполнения данной лабораторной работы была реализована U-Net-подобная нейронная сеть для сегментации изображений GTA5 с помощью трансферного обучения. Архитектура сети имеет предобученные слои сети VGG16. На изображении имеется несколько классов объектов: люди, дороги, деревья, дома и так далее. Каждый класс объектов отмечен определенным цветом.